

Лабораторна робота №1

Тема: ПРИХОВУВАННЯ ДАНИХ У ЗОБРАЖЕННІ МЕТОДОМ ЗАМІНИ НАЙМЕНШОГО ЗНАЧУЩОГО БІТА (LSB)

Мета: дослідити принципи роботи методу LSB на основі навчальної програми CrypTool 2; проаналізувати ємність стеганографічного приховування даних; реалізувати алгоритм LSB для приховування текстових даних у зображенні, а також функціонал для вилучення прихованого тексту з зображення.

Хід роботи:

Завдання 1. Виконати приховування повідомлення у зображенні методом LSB на основі навчальної програми CrypTool 2 (Templates⇒Steganography⇒Image Steganography with LSB).

1. Завантажте та встановіть програмне забезпечення CrypTool 2. CrypTool 2 (CT2) – це безкоштовна програма з відкритим вихідним кодом для Windows, призначена для навчання криптографії та криптоаналізу, а також стеганографії.



Рис. 1.1. Запущений CrypTool 2

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.07.000 – Лр.1			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Незнайко В.К.			Звіт з лабораторної роботи №1		Літ.	Арк.
Перевір.		Щур Н.О.						1
Керівник							ФІКТ Гр. ВТк-24-1	
Н. контр.								
Зав. каф.		Морозов А.В.						
							9	

2. Відкрийте проект Image Steganography with LSB (рис. 1.1), введіть своє ім'я та прізвище у поле для секретного повідомлення. Також, у відповідному полі, оберіть зображення, що буде слугувати контейнером (формат рекомендується обрати BMP або PNG (тільки не з прозорим фоном)).

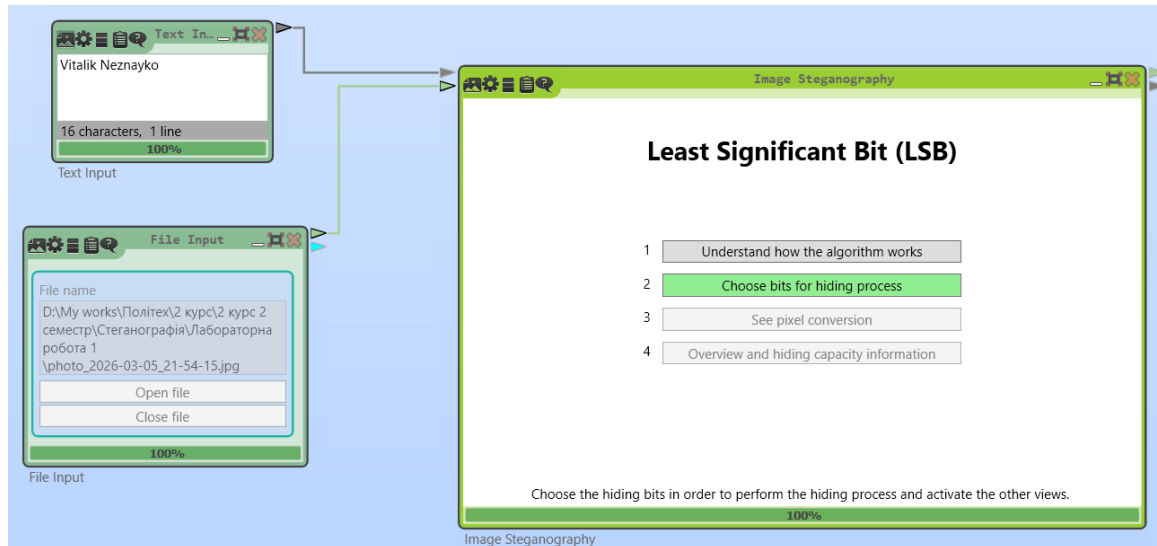


Рис. 1.1. Image Steganography with LSB

3. Запустіть програму на виконання (натисніть Play у головному меню програми) та оберіть кількість бітів для модифікації за допомогою опції Choose bits for hiding process. Оберіть один найменш значущий біт та настаніть Apply changes (рис. 1.2). Після цього дані вбудуються у контейнер та згенерується нове зображення зі стеганографічними даними (додайте скріншот до звіту).

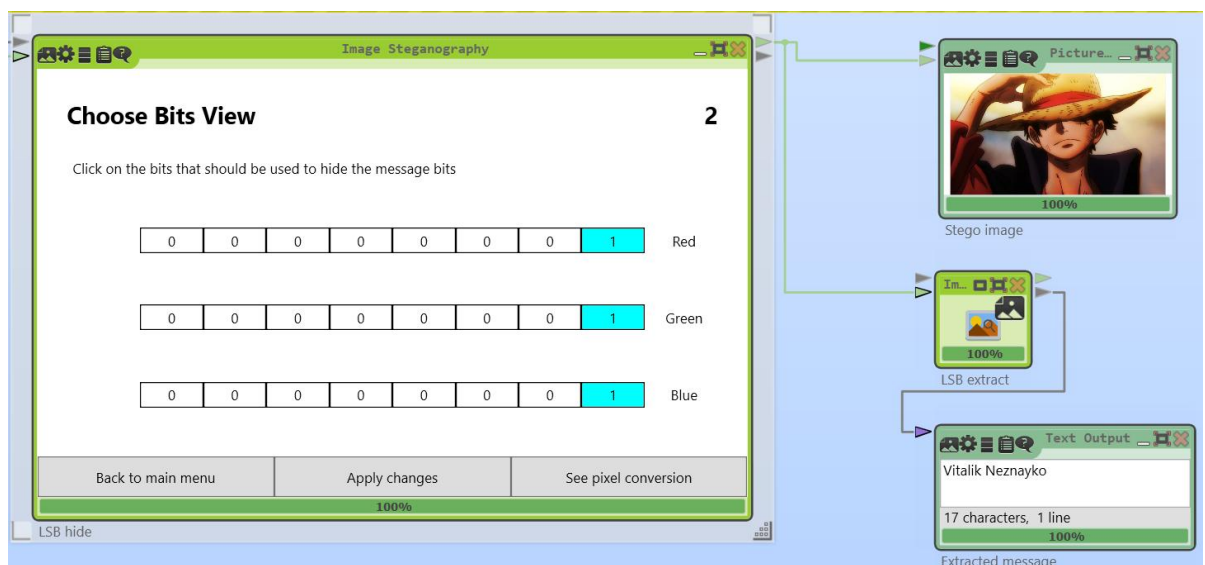


Рис. 1.3. Результат виконання

		Незнайко В.К.			ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.07.000 – Лр.1	Арк.
		Щур Н.О.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

4. Натисніть опцію See pixel conversation та перегляньте процес заміни пікселів. Детально опишіть у звіті приховування методом LSB першої та останньої літери секретного повідомлення (зі скріншотами). Зразок наведено нижче.

Перша літера V = 86 -> 01010110, тоді в зворотньому порядку 011 010 10

Остання літера o = 111 -> 01101111, тоді в зворотньому порядку 111 101 10

Перша літера	Остання літера
Модифікація пікселя (0,0)  RGB bits before RGB bits after Red value: 11100100 Red value: 11100100 Green value: 11100101 Green value: 11100101 Blue value: 11010101 Blue value: 11010101	Модифікація пікселя (40,0)  RGB bits before RGB bits after Red value: 11110001 Red value: 11110001 Green value: 11110010 Green value: 11110011 Blue value: 11100010 Blue value: 11100011
Модифікація пікселя (1,0)  RGB bits before RGB bits after Red value: 11101111 Red value: 11101110 Green value: 11110000 Green value: 11110001 Blue value: 11100000 Blue value: 11100000	Модифікація пікселя (41,0)  RGB bits before RGB bits after Red value: 11110001 Red value: 11110001 Green value: 11110010 Green value: 11110010 Blue value: 11100010 Blue value: 11100011
Модифікація пікселя (2,0)	Модифікація пікселя (42,0)

			
RGB bits before	RGB bits after	RGB bits before	RGB bits after
Red value: 1111001 1	Red value: 1111001 1	Red value: 1111000 1	Red value: 1111000 1
Green value: 1111010 0	Green value: 1111010 0	Green value: 1111001 0	Green value: 1111001 0
Blue value: 1110010 0	Blue value: 1110010 1	Blue value: 1110001 0	Blue value: 1110001 0

5. За допомогою опції Overview and Hiding Capacity Information перегляньте інформацію про довжину повідомлення та ємність приховування (додайте скріншот до звіту). Самостійно обчисліть довжину повідомлення та максимальну ємність приховування, якщо замінювати 1 найменш значущий біт. Порівняйте власноруч обчислені значення із тими, що на скріншоті (рис. 1.3).

Повідомлення Vitalik Neznayko містить 16 символів (включно з пробілом). Кожен символ кодується у 8 бітів. Таким чином довжина повідомлення:

$$16 * 8 = 128 \text{ бітів.}$$

Наше зображення розміром 1280x720. Тоді максимальна ємність приховування, якщо замінювати 1 найменш значущий біт буде:

$$C = 1280 * 720 * 3 * 1 = 2764800 \text{ бітів.}$$

Overview and Hiding Capacity Information

4

Message information:

Message length: 128 bit

Image information:

Image width: 1280

Image height: 720

Number of bits chosen to hide the secret message: 3

Hiding capacity: 2764800 bit

Hiding capacity percentage: 12,5 %



Back to main menu

		Незнайко В.К.			ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.07.000 – Лр.1	Арк.
		Щур Н.О.				4
Змн.	Арк.	№ докум.	Підпис	Дата		

Рис. 1.4. Інформація про довжину повідомлення та ємність приховування

6. Забезпечте збереження стегоконтейнеру до нового файлу. Для цього додайте до проекту компонент File Output, використовуючи розділ Tools на панелі модульних компонентів Components. Ця панель розміщена ліворуч робочого вікна та дає змогу додавати до проекту нові складові, таким чином удосконалюючи його.

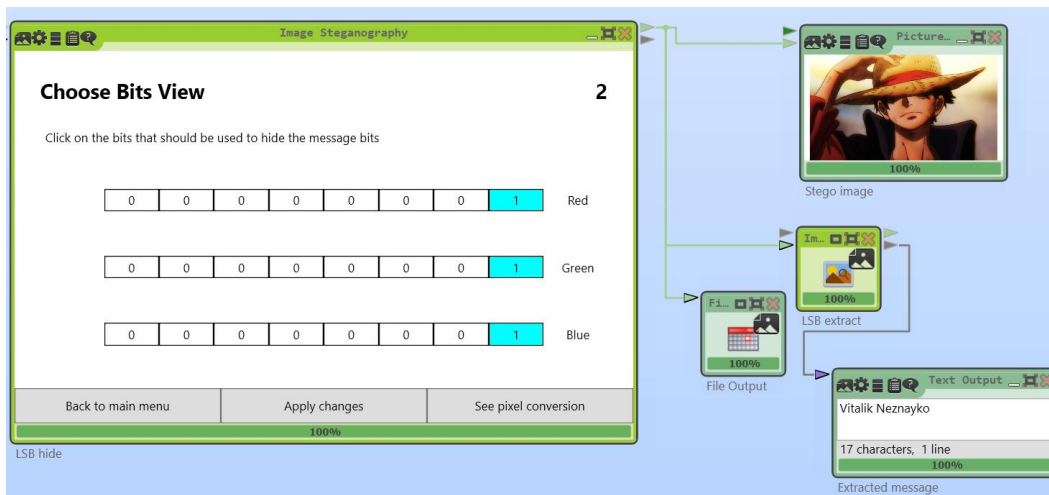


Рис. 1.5. Реалізація збереження нового файлу

7. Знову запустіть програму на виконання. Порівняйте у звіті оригінальне зображення та зображення із прихованими даними.



Рис. 1.6. Оригінальне зображення



Рис. 1.7. Зображення із прихованими даними

8. Обміняйтеся зображенням із вбудованими даними із іншим студентом своєї групи. Додайте отримане зображення до звіту.



Рис. 1.8. Зображення друга

9. Створіть новий проект для вилучення прихованого повідомлення, наприклад під назвою LSB_dec.cwm. Додайте до проекту компоненти File Input, Image Steganography та Text Output. Встановіть зв'язки, налаштуйте параметри роботи складових проекту як показано на рисунку (рис. 1.5).

10. Збережіть проект та перевірте його роботу. Результатом правильного виконання буде коректно вилучене повідомлення (рис. 1.6). Додайте скріншот проекту до звіту.

		Незнайко В.К.			ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.07.000 – Лр.1	Арк.
		Щур Н.О.				
Змн.	Арк.	№ докум.	Підпис	Дата		6

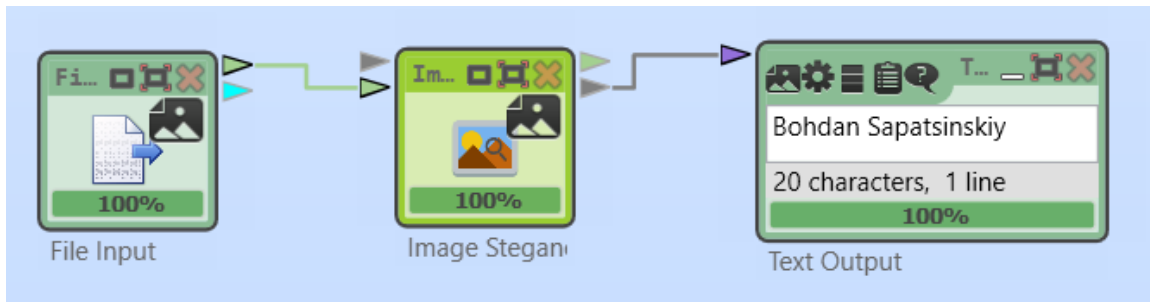


Рис. 1.9. Приховане повідомлення від друга

Завдання 2. Розробити програму мовою програмування (Python, C++, Java тощо) для приховування текстових даних у зображенні методом LSB. Реалізувати функціонал для вилучення прихованого тексту із зображення. Виконати порівняння оригінального зображення та зображення із прихованими даними.

Реалізувати інтерфейс взаємодії з користувачем – пропонувати вибір між приховуванням та вилученням тексту, запитувати текст для приховування та шлях до зображення. Забезпечити роботу із зображеннями у форматі PNG або BMP. Перевірити коректність роботи програми на різних зображеннях.

Лістинг:

```
from PIL import Image
import numpy as np

def text_to_binary(text):
    return ''.join(format(ord(c), '08b') for c in text)

def binary_to_text(binary):
    chars = []
    for i in range(0, len(binary), 8):
        byte = binary[i:i+8]
        chars.append(chr(int(byte, 2)))
    return ''.join(chars)

def hide_text(image_path, text, output_path):
    image = Image.open(image_path)
    image = image.convert("RGB")
    pixels = np.array(image)

    binary_text = text_to_binary(text) + "1111111111111110"

    flat_pixels = pixels.flatten()

    if len(binary_text) > len(flat_pixels):
        print("Текст занадто великий для цього зображення")
        return
```

```

for i in range(len(binary_text)):
    flat_pixels[i] = (flat_pixels[i] & 254) | int(binary_text[i])

new_pixels = flat_pixels.reshape(pixels.shape)

new_image = Image.fromarray(new_pixels.astype('uint8'), 'RGB')
new_image.save(output_path)

print("Текст успішно приховано у:", output_path)

def extract_text(image_path):

    image = Image.open(image_path)
    pixels = np.array(image)

    flat_pixels = pixels.flatten()

    binary_data = ""

    for pixel in flat_pixels:
        binary_data += str(pixel & 1)

    end_marker = "1111111111111110"

    index = binary_data.find(end_marker)

    if index != -1:
        binary_data = binary_data[:index]

    text = binary_to_text(binary_data)

    return text

def compare_images(original_path, stego_path):

    img1 = Image.open(original_path).convert("RGB")
    img2 = Image.open(stego_path).convert("RGB")

    img1 = np.array(img1)
    img2 = np.array(img2)

    difference = np.sum(img1 != img2)

    print("Кількість змінених значень пікселів:", difference)

def main():
    while True:
        print("\n1 - Приховати текст")
        print("2 - Витягнути текст")

```

		Незнайко В.К.			ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.07.000 – Лр.1	Арк.
		Щур Н.О.				8
Змн.	Арк.	№ докум.	Підпис	Дата		


```

print("3 - Порівняти зображення")
print("0 - Вийти")

choice = input("Ваш вибір: ")

if choice == "1":
    image_path = input("Шлях до зображення (PNG/BMP): ")
    text = input("Введіть текст для приховування: ")
    output = input("Назва нового файлу: ")

    hide_text(image_path, text, output)

elif choice == "2":
    image_path = input("Шлях до зображення: ")

    text = extract_text(image_path)

    print("Прихований текст:")
    print(text)
elif choice == "3":
    original = input("Оригінальне зображення: ")
    stego = input("Зображення з текстом: ")
    compare_images(original, stego)
elif choice == "0":
    break
else:
    print("Невірний вибір")
if __name__ == "__main__":
    main()

```

Результат виконання:

		Незнайко В.К.			ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.07.000 – Лр.1	Арк.
		Щур Н.О.				
Змн.	Арк.	№ докум.	Підпис	Дата		9

```

1 - Приховати текст
2 - Витягнути текст
3 - Порівняти зображення
0 - Вийти
Ваш вибір: 1
Шлях до зображення (PNG/BMP): bird.png
Введіть текст для приховування: Hello world!
Назва нового файлу: bird_with_secret.png
Текст успішно приховано у: bird_with_secret.png

1 - Приховати текст
2 - Витягнути текст
3 - Порівняти зображення
0 - Вийти
Ваш вибір: 2
Шлях до зображення: bird_with_secret.png
Прихований текст:
Hello world!

1 - Приховати текст
2 - Витягнути текст
3 - Порівняти зображення
0 - Вийти
Ваш вибір: 3
Оригінальне зображення: bird.png
Зображення з текстом: bird_with_secret.png
Кількість змінених значень пікселів: 56

1 - Приховати текст
2 - Витягнути текст
3 - Порівняти зображення
0 - Вийти
Ваш вибір: 0

```

Рис. 1.10. Результат виконання

Висновок: В ході виконання лабораторної роботи дослідили принципи роботи методу LSB на основі навчальної програми CrypTool 2; проаналізували ємність стеганографічного приховування даних; реалізували алгоритм LSB для приховування текстових даних у зображенні, а також функціонал для вилучення прихованого тексту з зображення.

		Незнайко В.К.			ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.07.000 – Лр.1	Арк.
		Щур Н.О.				10
Змн.	Арк.	№ докум.	Підпис	Дата		